

MicroInvest

A Micro Investment Platform

Ji Chengyu Student ID G2608005K

SC6113 Individual Assignment Financial Decentralized Application

7 October 2026

1 Introduction

MicroInvest is an educational financial DApp for investing beginners. Users deposit Sepolia test ETH into one pool, receive internal shares, inspect their position and redeem principal at any time. A Solidity contract connects to an English interface and Flask backend, deployed at [MicroInvest on Render](#).

One ETH equals one share, including fractions. Funds remain in the contract for withdrawals, with no market investment, yield, platform fee or administrator. Users pay network gas separately. The prototype demonstrates ownership, accounting and confirmation without suggesting investment returns.

2 Problem Statement

A beginner must distinguish contributions, ownership and transaction cost, as well as wallet approval from blockchain confirmation. Unclear balances or withdrawal conditions make it difficult to understand where funds are held and how to recover them.

MicroInvest addresses this through fractional deposits, visible share conversion and partial or full redemption. On-chain state determines each account's entitlement; confirmed events provide history. Market allocation, diversification and capital growth are outside the prototype's scope.

3 Objectives

The objectives are to implement exact fractional ownership in Solidity; enable MetaMask connection and account selection; support deposits and caller-bound principal redemption; provide useful Flask queries for positions, events and receipts; and distinguish validation errors, wallet rejection, pending transactions and mined results. Evaluation covers contract authorization, accounting, common failure paths, interface behavior and measured gas. The source, deployment details, README and report follow the assignment requirements [1].

4 System Architecture

The application separates reads from signed writes. The browser obtains public configuration, pool state, individual positions and transaction history from Flask. Web3.py queries the configured Sepolia JSON-RPC service. Deposits and redemptions are constructed with ethers and submitted through MetaMask after user approval. The backend does not sign transactions or hold the user's private key.

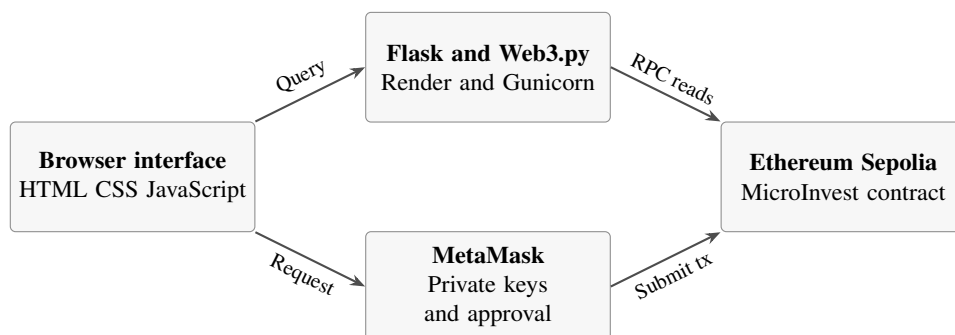


Figure 1: System architecture and the boundary between queries and wallet signing.

The contract is the source of truth. No database is used; Flask reconstructs confirmed balances and events from the chain after restart. Browser storage retains transaction hashes and preferred accounts per pool/network but cannot grant permissions or establish balances.

Every blockchain API query verifies the configured chain and exact deployed runtime bytecode. Pool and position calls use a snapshot block. History scans are bounded to 5,000 blocks per request, with log requests split into chunks of at most

1,000 blocks. An exclusive block/log-index cursor supports older pages without repeating events when several records share a block.

5 Technologies Used

Table 1: Technologies and their roles

Layer	Technology	Role
Contract	Solidity 0.8.30	Custody, shares, caller authorization and events
Development tools	Node.js 24, Hardhat 3, pnpm	Compilation, local chain, contract and browser tests
Browser	HTML, CSS, JavaScript, ethers 6	Responsive pages and wallet transaction construction
Backend	Python 3.12, Flask, Web3.py	Validated read-only blockchain APIs
Wallet and network	MetaMask, Ethereum Sepolia	User approval, signing and testnet execution
Hosting and workflow	Render, Unicorn, GitHub, VS Code	Web hosting, source control and development

Node.js supports development and testing; the deployed service runs Python and Unicorn. Committed artifacts and the ethers bundle avoid Solidity compilation or Node.js installation during the web build. Unicorn supplies the production WSGI server [5, 6].

6 Smart Contract Design

`MicroInvest.sol` stores `shares[address]` and `totalShares` as unsigned integer units. One displayed share has 18 decimal places, so a deposit’s wei value directly equals the credited share units. This avoids rounding in conversion. The supported accounting invariants are that total shares equal the sum of user shares, each user’s redeemable wei equals their shares, and contract balance is at least the recorded principal.

Table 2: Contract methods and validation

Method	State transition	Validation
<code>deposit()</code>	Credit the caller and total shares by <code>msg.value</code>	Deposit is greater than zero
<code>withdraw(amountWei)</code>	Debit caller and total; send the same amount to caller	Positive amount and sufficient caller shares
<code>withdrawAll()</code>	Redeem the caller’s entire nonzero position	Caller has shares
<code>shares(address)</code> and <code>totalShares()</code>	Read ledger balances	No state change

For example, depositing 0.003 ETH credits 0.003 shares. Redeeming 0.001 debits exactly that amount, leaving 0.002. Full redemption then clears the position. Gas is charged to the wallet separately, so redeeming all principal does not restore gas previously spent.

Authorization follows `msg.sender`: a transaction changes only the calling account’s entitlement. Shares cannot be transferred. The deploying account receives no withdrawal privilege, and the contract has no owner, pause or upgrade function. These decisions keep the participant’s rules explicit, while also removing administrative recovery options.

Withdrawals check entitlement, debit state and then transfer ETH. A reentrancy lock blocks nested entry; a failed transfer reverts the entire operation, including the ledger debit. This follows the checks-effects-interactions principle described in Solidity’s security guidance [2]. Tests use both a receiver that refuses ETH and a receiver that attempts a nested withdrawal.

Deposited and Withdrawn events contain an indexed investor, the wei amount and resulting shares. Successful events support confirmed history. A wallet rejection or reverted operation creates no successful business event. ETH forcibly received outside `deposit()` is reported as excess balance and creates no new shares or yield.

The compiler optimizer is enabled with 200 runs. Contract operations change a fixed amount of storage and do not loop over investors. Gas measurements are reported in Section 9; they are evidence from specific transactions rather than a guarantee of future cost.

7 Application Design

The dark English overview explains the testnet, fixed share conversion, zero platform fee and separate gas cost. It combines deposit/redemption controls with personal shares, redeemable principal and wallet balance. Separate controls support partial redemption and full exit. Labels, visible feedback and responsive layouts support the intended beginner audience.

A connected wallet address opens an account menu on both the overview and activity page. The list uses addresses actually exposed by `eth_accounts`. Manage accounts requests wallet permission after a user click; selecting an account refreshes its position and history. Before signing, the application confirms that the selected account is still authorized and uses `getSigner(selectedAddress)`, even when it is not the first address returned. Provider account/network events and permission requests follow the interfaces in [3, 4].

The overview shows the five newest confirmed records. View all activity opens a separate page that initially shows 20 records and loads older cursor pages. Links point to actual Sepolia transactions. The complete history page also accepts a public wallet address for read-only viewing.

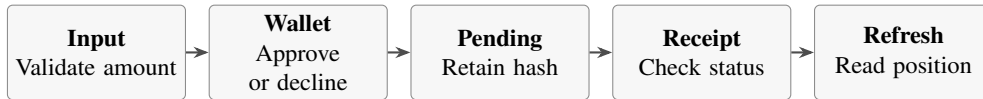


Figure 2: Transaction feedback distinguishes wallet approval from a mined result.

A rejected wallet request is shown as declined. A submitted hash is pending until a receipt is available. A successful receipt refreshes the position; a failed receipt reports a revert. If confirmation is delayed, the hash is retained for later checking. Account changes clear previous data, and older asynchronous responses cannot overwrite the new account’s position.

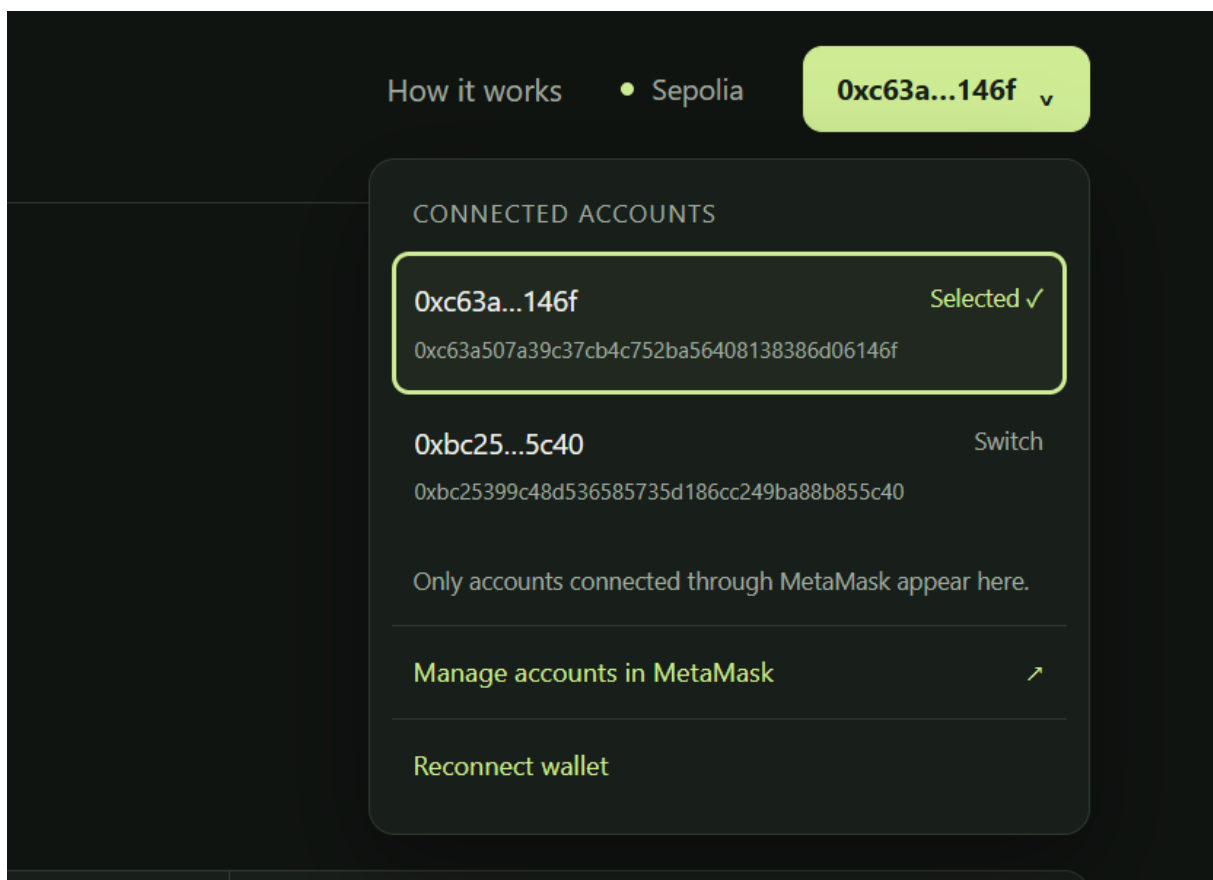


Figure 3: Connected MetaMask account on Sepolia with a second authorized account available in the account menu.

8 Implementation

The repository contains the business contract and adversarial tests, an exported ABI/bytecode artifact, Flask routes, the blockchain query service, HTML templates and JavaScript modules. The browser actively calls `/api/config`, `/api/pool`, `/api/position/<wallet>`, `/api/history/<wallet>` and `/api/transactions/<hash>`. Flask validates addresses, hashes and pagination parameters and returns amounts as decimal strings. JavaScript uses `BigInt` for wei rather than floating-point arithmetic.

The MetaMask deployment page lets the user deploy without supplying a private key to Flask. Public deployment metadata records the address and deployment block. On Render, the build installs `requirements.txt` and Gunicorn starts `app:app`. Environment configuration identifies Sepolia, its RPC endpoint and this deployed pool; `/healthz` checks process readiness, while `/api/pool` verifies real blockchain access.

9 Testing and Results

Automated testing combines contract tests, backend unit tests, exact-amount tests and an actual browser against Flask and a local Hardhat chain. The browser's EIP-1193 wallet is simulated. Backend unit tests mock the RPC boundary; the integration suite supplies real local state changes. These results do not by themselves prove real MetaMask behavior or hosted operation.

Table 3: Automated testing results

Suite	Result	Evaluation covered
Contract	12 passed, 6 October	Exact units, isolation, transfer rollback, reentrancy and a 40-operation accounting sequence
Flask backend	17 passed, 7 October	Validation, public configuration, bytecode/network checks, receipt states and cursor boundaries
Amount and error handling	4 passed, 7 October	One wei, large integers, invalid decimals and safe wallet errors
Browser integration	19 checks passed, 7 October	Deposit/exit flow, account selection/signing, five-record preview, pagination, mobile layout and RPC failure

For invalid input, zero, negative, exponential, nonnumeric and over-precision values are rejected. Excess redemption cannot spend another account's shares. Refusing a wallet request leaves contract state unchanged. Simulated network changes disable writes; an RPC outage displays an unavailable position instead of inventing a zero balance. At a 390-pixel viewport, the document and account menu remain within the screen width.

I completed functional testing on Render with MetaMask. Independent read-only checks on 7 October returned HTTP 200 for the home, configuration, health and pool endpoints, identifying chain 11155111 and the expected contract. That snapshot showed a solvent pool with zero principal and eight confirmed participant events. A later history screenshot contains nine records, including a subsequent 0.04 ETH deposit. The supplied API snapshot shows 0.04 ETH principal and balance at block 11857366, with zero excess and solvent: true. The later confirmed redemption at block 11857372 shows zero personal shares and redeemable principal. These screenshots document different blocks rather than conflicting balances.

Table 4: Confirmed Sepolia principal lifecycle

Observed Sepolia operation	Amount in ETH	Shares after	Block
Deposit	0.003	0.003	11856700
Partial redemption	0.001	0.002	11856703
Full redemption	0.002	0	11856705

All three receipts report success. Their full transaction hashes and raw API evidence are retained in docs/evidence/live-validation-2026-10-07.json. The chain record proves the debit/credit sequence; screenshots illustrate the corresponding user experience.

Table 5: Gas used in separate local and Sepolia observations

Operation	Local receipt gas	Observed Sepolia receipt gas
Deposit	71,884	245,994
Partial redemption	50,282	53,700
Full redemption	50,272	50,912

These are separate observations with different transaction amounts and execution environments. The local cases used 0.01 ETH for the first deposit, 333 wei for partial exit and 999 wei for full exit. Actual fee is receipt gas multiplied by effective gas price. The three Sepolia transactions cost 0.000622855710866760, 0.000132064592365200 and 0.000127495073925600 ETH respectively. No throughput benchmark or novice-user study was conducted.

Deployment and live evidence

The pool is deployed at 0xE560121978f80c390f6B0d091E9579A2812Cb3DD on Sepolia. Its creation receipt is successful in block 11856501 and used 1,814,581 gas. Exact runtime bytecode matches the committed artifact. The deploying account is an ordinary participant.

[Deployment transaction](#) | [Deposit transaction](#) | [Partial redemption](#) | [Full redemption](#)

All your activity

Your confirmed deposits and redemptions, newest first. Load older activity to browse the complete history.

Transaction history

Viewing activity for 0xc63a507a39c37cb4c752ba56408138386d06146f

Refresh

Action	Amount	Date	Status	Transaction
➤ Deposit	0.04 ETH	7 Oct 2026, 01:29	Confirmed	0x3cb2...61f8
✓ Redemption	0.03 ETH	7 Oct 2026, 00:00	Confirmed	0x039d...4be8
✓ Redemption	0.03 ETH	6 Oct 2026, 23:59	Confirmed	0xd88e...741d
➤ Deposit	0.04 ETH	6 Oct 2026, 23:59	Confirmed	0xcbc9...dcb0
✓ Redemption	0.02 ETH	6 Oct 2026, 23:57	Confirmed	0x4c20...41fa
➤ Deposit	0.04 ETH	6 Oct 2026, 23:56	Confirmed	0x254a...3726
✓ Redemption	0.002 ETH	6 Oct 2026, 23:20	Confirmed	0xe47a...6e9a
✓ Redemption	0.001 ETH	6 Oct 2026, 23:20	Confirmed	0xcb3e...804e
➤ Deposit	0.003 ETH	6 Oct 2026, 23:19	Confirmed	0x0304...0798

Figure 4: Complete activity page showing confirmed deposits and redemptions, including the 0.003 ETH deposit, 0.001 ETH partial redemption and 0.002 ETH full exit.

```
{
  "blockNumber": 11857366,
  "contractBalanceWei": "40000000000000000",
  "excessWei": "0",
  "principalEth": "0.04",
  "principalWei": "40000000000000000",
  "solvent": true,
  "totalShares": "0.04",
  "totalSharesUnits": "40000000000000000"
}
```

Figure 5: Pool API at block 11857366: 0.04 ETH principal and contract balance, zero excess and solvent true, before the later redemption.

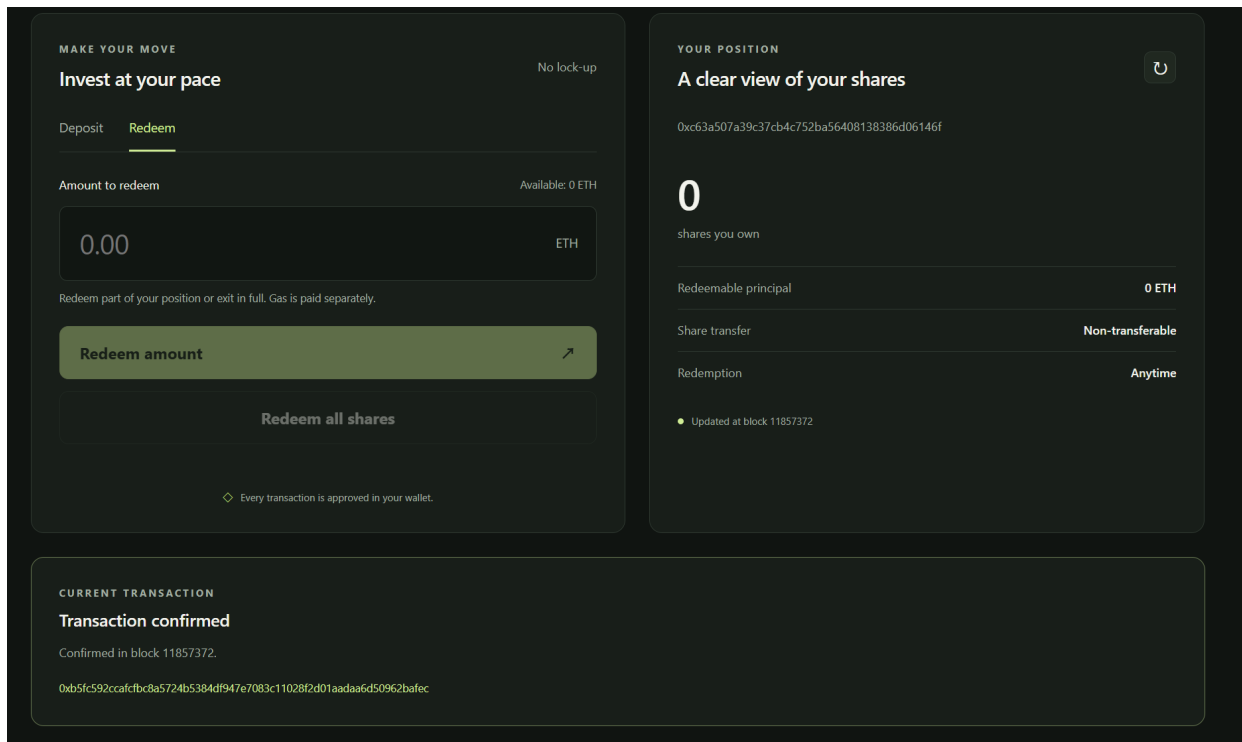


Figure 6: Confirmed redemption at block 11857372 with zero personal shares and zero redeemable principal.

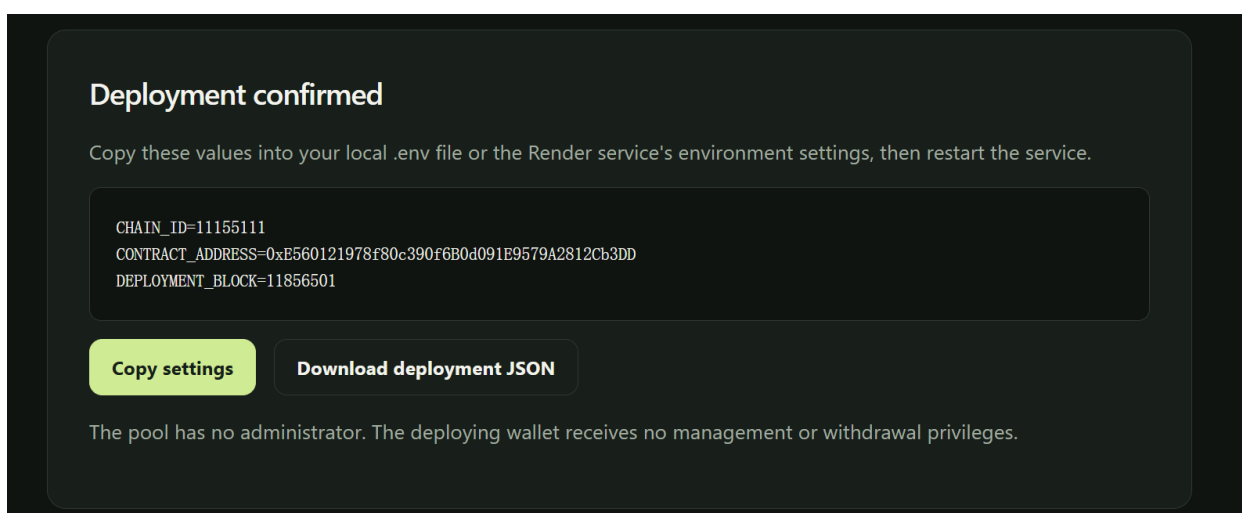


Figure 7: Deployment confirmation showing Sepolia chain ID 11155111, the deployed contract address and block 11856501.

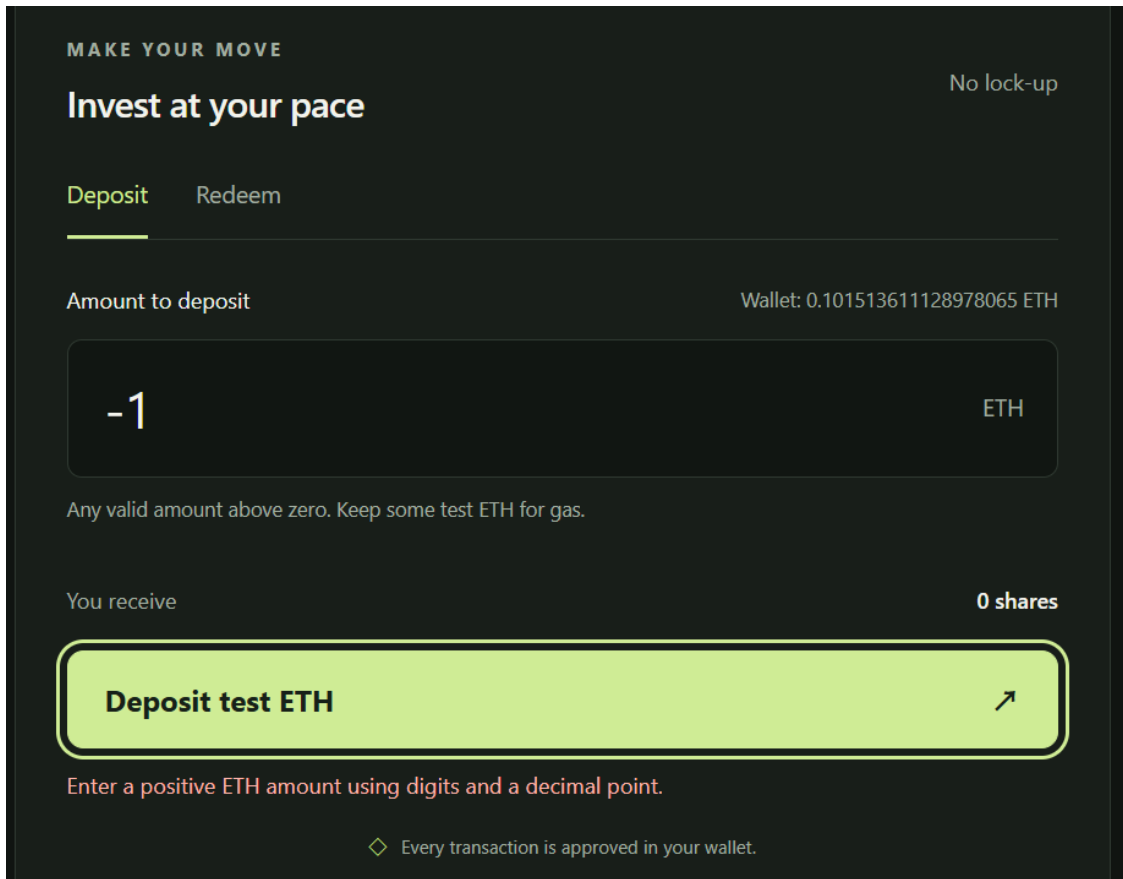


Figure 8: A negative deposit amount of -1 ETH with a message requiring a positive ETH amount.

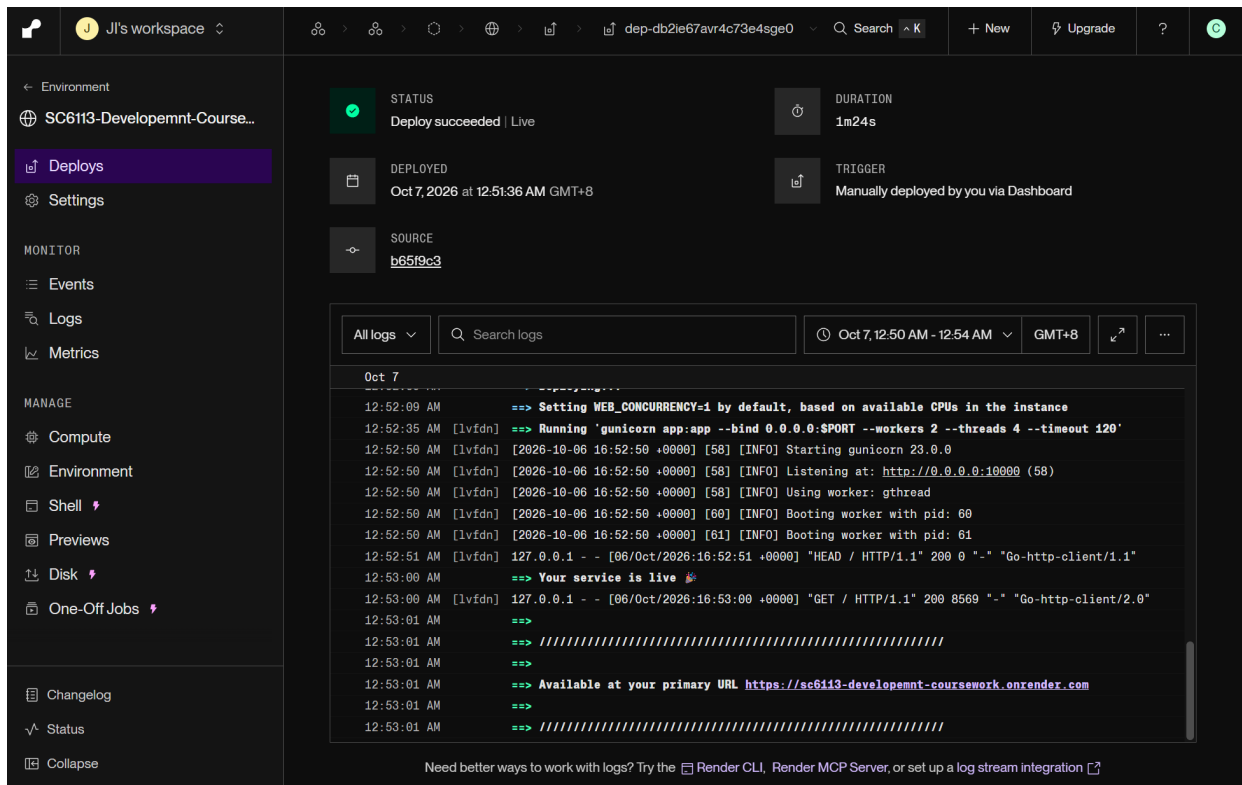


Figure 9: Render deployment succeeded and Live for commit b65f9c3, with Gunicorn startup logs and the public service URL.

The interface validates exact amounts and wallet context; Solidity independently enforces positive values and caller ownership. Flask rejects malformed query parameters and exposes no write endpoint. Private keys stay in MetaMask; RPC credentials stay server-side. Sanitized errors and a Content Security Policy reduce exposure and unsafe content.

Addresses, events and amounts remain public despite the absence of a database. No identity profiles are collected. The interface explains test ETH and the absence of returns. Usability observations come from functional checks and my walkthrough; no controlled beginner study was conducted.

10 Challenges Encountered

Integer wei and string responses solved cross-layer decimal precision. Separate signing, pending, mined and unknown states prevented premature success messages. Bounded history queries and block/log-index cursors handled RPC limits and same-block events. Account selection required the selected authorized signer; separating public artifacts from runtime secrets kept Render deployment reproducible.

11 Limitations

The prototype holds principal without a market strategy, diversification or yield. One mined confirmation is sufficient for its feedback but is not chain finality. Public RPC availability and wallet behavior affect operation, and old inactive history can require several page requests. There is no formal external security audit, load test or beginner usability study. With no administrator or upgrade mechanism, there is no administrative rescue path for excess ETH. The testnet prototype is not evaluated for custody of real savings.

12 Future Improvements

Future work includes beginner usability testing, clearer delayed-transaction recovery, an event index with reorganization handling and multiple wallet providers. Real investment strategies, transferable tokens or additional assets require a separate specification, risk model and security review.

13 Conclusion

MicroInvest implements the selected micro-investment workflow through exact fractional shares, wallet-authorized transactions and caller-bound principal exit. Flask provides real contract queries and event history, while MetaMask signs writes. Automated tests cover accounting and failure cases, and the deployed Sepolia/Render integration supports the demonstrated lifecycle. The result meets the financial DApp learning objectives within a clearly stated educational scope.

References

- [1] SC6113 Individual Assignment Financial Decentralized Application. Supplied coursework Word document, Sections III and VI–XI.
- [2] Solidity 0.8.30. *Security Considerations*. <https://docs.soliditylang.org/en/v0.8.30/security-considerations.html>.
- [3] Ethereum Improvement Proposals. *EIP-1193 Ethereum Provider JavaScript API*. <https://eips.ethereum.org/EIPS/eip-1193>.
- [4] Ethereum Improvement Proposals. *EIP-2255 Wallet Permissions System*. <https://eips.ethereum.org/EIPS/eip-2255>.
- [5] Flask 3.1 documentation. *Deploying to Production*. <https://flask.palletsprojects.com/en/stable/deploying/>.
- [6] Render documentation. *Deploy a Flask App*. <https://render.com/docs/deploy-flask>. Online documentation accessed 7 October 2026.